

Discriminative and Generative Neuronal Networks on FSD Break Data

Dataset and Task

❖ Dataset:

- consists of **multi-channel time series data**
- each sample containing acceleration and rotation readings from different axes
- along with a class label indicating the braking state.

❖ Goal:

accurately **classify these braking states** using deep learning methods.

FCN

Fully Convolutional Network

Model:

❖ FCN:

- **Input** (1D Data) →
 - **Conv1D Layer 1** (input_channels → 64, kernel=8, ReLU) →
 - **Conv1D Layer 2** (64 → 128, kernel=5, ReLU) →
 - **Conv1D Layer 3** (128 → 64, kernel=3, ReLU) →
 - **Global Average Pooling** (output size: 1) →
 - **Dropout Layer** (p=0.3) →
 - **Fully Connected Layer** (64 → num_classes) →
 - **Output** (Class Scores)
-
- The FCN model processes 1D input data through three **convolutional layers**—each followed by **ReLU activation**—extracting increasingly abstract features.
 - **Global average pooling** reduces the sequence to a single value per feature map, followed by **dropout** for regularization.
 - Finally, a fully connected layer produces the classification output.

```
# FCN Model
class FCN(nn.Module):
    def __init__(self, input_channels, num_classes, dropout_p=0.3):
        super(FCN, self).__init__()
        self.conv1 = nn.Conv1d(input_channels, 64, kernel_size=8)
        self.conv2 = nn.Conv1d(64, 128, kernel_size=5)
        self.conv3 = nn.Conv1d(128, 64, kernel_size=3)
        self.global_pool = nn.AdaptiveAvgPool1d(1)
        self.dropout = nn.Dropout(p=dropout_p)
        self.fc = nn.Linear(64, num_classes)

    def forward(self, x):
        x = torch.relu(self.conv1(x))
        x = torch.relu(self.conv2(x))
        x = torch.relu(self.conv3(x))
        x = self.global_pool(x).squeeze(-1)
        x = self.dropout(x)
        x = self.fc(x)
        return x
```

Main Variables and Settings

Main Variables and Settings

❖ Data Preprocessing

- **Normalization:**
MaxAbs Scaling
- **Standardization:**
z-score standardization
- **Number of Categories:**
2 or 12 categories
- **Data Augmentation**
 - Traditional: **Upsampling** (for class balance)
 - **Generative:**
GAN1/2, cGAN, dann, RNN, tGAN

❖ Model and Training

- **Regularization:** L2 regularization (0.01)
Dropout (0.3, 0.1, 0.5)
- **Batch Size:** 32, 64, or 128
- **Split Strategy:**
random split, K-Fold
- **Optimizer:**
Adam, learning rate (0.001, 0.0005, 0.005)
- **Loss Function:** CrossEntropyLoss,
class weights
- **Epochs:** 20, 30, 40
- **Early Stopping:**
early stopping (patience)

Observations

Observations | Data Processing

- **Normalization:** MaxAbs Scaling

- ❖ **What MaxAbs Scaling Does:**

- It scales each feature (column) of the data so that its maximum absolute value becomes 1.
 - This means all values will be between -1 and 1 for each feature.

- ❖ **When it works well:**

MaxAbs scaling works best for data that is roughly already centered and does not contain extreme outliers. It is not robust to outliers and does not center the data.

```
# MaxAbs Scaling
def maxabs_normalize(train, test):
    # Flatten into two dimensions for easy statistics
    train_2d = train.reshape(-1, train.shape[-1])
    max_abs = np.max(np.abs(train_2d), axis=0)
    max_abs[max_abs == 0] = 1.0
    train_norm = train / max_abs
    test_norm = test / max_abs
    return train_norm, test_norm
```

- ❖ **Experiment:**

		normalization	
data	MaxAbs Scaling	Maxabs	
	z_score standardize		z_score
classes	12/2 (remap)	12	12
Data augmentation	upsampling minority classes		
	Gan1		
	Gan2		
	cGAN		
	dann		
	rnn		
	tGAN		
FCN model	dropout(p)	0.3	
	early_stopping(patience)		
split?	random split		
	K-Folds		
Batch_size		32	
optimizer(adar)	lr	0.001	
	L2()	0.01	
loss fct(CrossE	Compute class weights	y	
Epochs		30	
	Val_Acc		0.9409
	Test_Acc	0.1049	0.3636

- ❖ **Result:**

With the given parameter settings, using MaxAbs normalization did not produce good results.

The validation accuracy was only 0.0444, and the test accuracy was 0.1049, both of which are very low.

Observations | Data Processing

- **Standardized: z-score** standardization

- ◆ **How it works:**

- It calculates the mean and standard deviation for each feature (column) in the training set.
 - Then, it subtracts the mean and divides by the standard deviation for both the training and test data.
 - This transformation makes each feature have a mean of 0 and a standard deviation of 1.

- ◆ **Why use it:**

Z-score standardization is useful when you want all features to be on the same scale, especially for algorithms sensitive to feature scaling. It helps models converge faster and improves their performance.

```
# z-score standardize
def zscore_normalize(train, test):
    # Flatten into two dimensions for easy statistics
    train_2d = train.reshape(-1, train.shape[-1])
    mean = np.mean(train_2d, axis=0)
    std = np.std(train_2d, axis=0)
    train_norm = (train - mean) / (std + 1e-8)
    test_norm = (test - mean) / (std + 1e-8)
    return train_norm, test_norm
```

- ◆ **Experiment:**

		normalization	
data	MaxAbs Scaling	Maxabs	
	z_score standardize		z_score
classes	12/2 (remap)	12	12
Data augmentation	upsampling minority classes		
	Gan1		
	Gan2		
	cGAN		
	dann		
	rnn		
	tGAN		
FCN model	dropout(p)	0.3	
	early_stopping(patience)		
split?	random split		
	K-Folds		
Batch_size		32	
optimizer(adar)	lr	0.001	
	L2()	0.01	
loss fct(CrossE	Compute class weights	γ	
Epochs		30	
	Val_Acc		0.9408
	Test_Acc	0.1049	0.3636

- ◆ **Result:**

Using z-score standardization significantly improved the results. The validation accuracy increased to 0.9408, and the test accuracy also improved to 0.3636.

Observations | Data Processing

```
# Remap labels to two categories: working (0) and non-working (1)
def remap_labels(y):
    return np.array([0 if label == 0 else 1 for label in y])
```

- **Number of Categories:** 2 or 12 categories

- ◆ **How it works:**

The remap_labels function takes the original labels for the four brakes (left, right, front, and rear), where each label can be one of 12 categories. It converts these into two categories:

- 0 for "working" (if the original label is 0)
- 1 for "non-working" (if the original label is any value other than 0)

- ◆ **Why use it:**

Originally, there were 12 different categories for each brake, representing different conditions of faults. By remapping to just two categories (working and non-working), the problem is simplified to binary classification. This makes it easier to train and interpret models, especially when the main concern is simply whether each brake is functioning or not.

- ◆ **Experiment:**

		normalization	
data	MaxAbs Scaling		
	z_score standardize	z_score	y
classes	12/2 (remap)	12	2
Data augmentation	upsampling minority classes		
	Gan1		
	Gan2		
	cGAN		
	dann		
	rnn		
	tGAN		
FCN model	dropout(p)	0.3	0.3
	early_stopping(patience)		
split?	random split		
	K-Folds		
Batch_size		64	64
optimizer(ada	lr		
	L2()		
loss fct(CrossE	Compute class weights		y
Epochs			30
	Val_Acc	0.9408	
	Test_Acc	0.3636	0.6434

- ◆ **Result:**

When the problem was changed from 12 classes to just 2 categories ("working" and "non-working"), the results improved significantly.

The validation accuracy reached 0.9938, and the test accuracy increased to 0.6434

- ◆ **Disadvantages and Potential Issues:**

- Loss of detailed information about fault types
- No fine-grained analysis or fault localization possible
- Rare classes become invisible
- Less actionable results for real-world applications
- Risk of label confusion and noise
- Possible overfitting due to task simplification

Observations | Model and Training

- **Data Augmentation - Traditional: Upsampling** (for class balance)

• **Upsampling (for class balance)** is a data augmentation technique used to address class imbalance in a dataset. When one class has much fewer samples than others, the model may not learn to recognize it well. Upsampling works by increasing the number of samples in the minority class, usually by randomly duplicating existing samples from that class. This helps balance the class distribution, allowing the model to learn from all classes more effectively and improving overall performance on imbalanced datasets.

```
# Upsampling to achieve class balance
max_count = max([np.sum(y_train == i) for i in range(num_classes)])
x_train_balanced = []
y_train_balanced = []
for i in range(num_classes):
    class_x = x_train[y_train == i]
    class_y = y_train[y_train == i]
    if len(class_x) == 0:
        continue # Theoretically, it should not occur.
    class_x_upsampled, class_y_upsampled = resample(
        class_x, class_y, replace=True, n_samples=max_count, random_state=42
    )
    x_train_balanced.append(class_x_upsampled)
    y_train_balanced.append(class_y_upsampled)
x_train = np.concatenate(x_train_balanced, axis=0)
y_train = np.concatenate(y_train_balanced, axis=0)
```

❖ Experiment:

data	MaxAbs Scaling		
	z_score standardize	y	y
classes	12/2 (remap)	2	2
Data augmentation	upsampling minority classes		unsampling
	Gan1		
	Gan2		
	cGAN		
	dann		
	rnn		
	tGAN		
FCN model	dropout(p)	0.3	0.3
	early_stopping(patience)	3min27	
split?	random split		
	K-Folds		
Batch_size		64	64
optimizer(adar)	lr	0.001	0.001
	L2()		
loss fct(CrossE	Compute class weights	y	y
Epochs		30	30
	Val_Acc		
	Test_Acc	0.6673	0.6314

❖ Result:

After applying upsampling, the accuracy dropped slightly, but it still remained above 0.6. The validation and test accuracies with and without upsampling are as follows:

- Without upsampling:

Val_Acc = 0.9889, Test_Acc = 0.6673

- With upsampling:

Val_Acc = 0.9945, Test_Acc = 0.6314

Observations | Model and Training

Data Augmentation - Generative: GAN1/2, cGAN, dann, RNN, tGAN

- **Generative data** augmentation methods like GANs, cGANs, DANNs, RNNs, and tGANs create synthetic data to enrich and balance datasets, helping improve model training, especially when real data is limited or imbalanced.

```
# Upsampling to achieve class balance
max_count = max([np.sum(y_train == i) for i in range(num_classes)])
x_train_balanced = []
y_train_balanced = []
for i in range(num_classes):
    class_x = x_train[y_train == i]
    class_y = y_train[y_train == i]
    if len(class_x) == 0:
        continue # Theoretically, it should not occur.
    class_x_upsampled, class_y_upsampled = resample(
        class_x, class_y, replace=True, n_samples=max_count, random_state=42
    )
    x_train_balanced.append(class_x_upsampled)
    y_train_balanced.append(class_y_upsampled)
x_train = np.concatenate(x_train_balanced, axis=0)
y_train = np.concatenate(y_train_balanced, axis=0)
```

❖ Experiment & Results

		without	gan1(7min)	gan2(13min)	cgan(17min)	dann(12min)	rnn(8min)	tgan(13min)	rnn(8min)+unsampling
	Val Acc								
	Test Acc	0.6673	0.5874	0.5544	0.6404	0.5185	0.6454	0.6234	0.6424

The original data produced the best results. However, using cGAN and RNN achieved performance very close to the best result. DANN performed the worst among the generative methods. Combining RNN with upsampling produced similar results to using RNN alone.

The models ranked from best to general performance are:

1. Original data (without augmentation)
2. RNN $\approx$ RNN + upsampling $\approx$ cGAN
3. tGAN
4. GAN
5. DANN

Observations | Model and Training

```
class FCN(nn.Module):
    def __init__(self, input_channels, num_classes, dropout_p=0.3):
        self.dropout = nn.Dropout(p=dropout_p)
```

• Regularization: L2 regularization (0.01) Dropout (0.3, 0.1, 0.5)

- The dropout probability (dropout_p) determines **how many neurons are randomly deactivated** during training in each iteration.
- **Smaller dropout values** (e.g., 0.1 or 0.2) mean fewer neurons are dropped. This allows the network to retain more information but may not be as effective at preventing overfitting.
- **Larger dropout values** (e.g., 0.5 or higher) mean more neurons are dropped. This increases regularization, making the model less likely to overfit, but if the value is too high, it can hurt the model's ability to learn important patterns and reduce overall performance.
- Choosing an appropriate dropout value is important. A moderate value (such as 0.3) is often used as a starting point, and should be adjusted based on validation performance.

❖ Experiment:

		dropout 0.1/0.5		
data	MaxAbs Scaling			
	z_score standardize	y	y	y
classes	12/2 (remap)	2	2	2
Data augmentation	upsampling minority classes			
	Gan1			
	Gan2			
	cGAN			
	dann			
	rnn			
	tGAN			
FCN model	dropout(p)	0.3	0.1	0.5
	early_stopping(patience)			
split?	random split			
	K-Folds			
Batch_size		64	64	64
optimizer(ada)	lr			
	L2()			
loss fct(CrossE)	Compute class weights	y	y	y
Epochs		30	30	30
	Val Acc			
	Test Acc	0.6434	0.6454	0.6444

❖ Result:

Using different dropout values (0.3, 0.1, and 0.5) did not lead to significant differences in the results. The validation and test accuracies for these three settings were as follows:

- Dropout 0.3: Val_Acc = 0.9938, Test_Acc = 0.6434
- Dropout 0.1: Val_Acc = 0.9914, Test_Acc = 0.6454
- Dropout 0.5: Val_Acc = 0.9551, Test_Acc = 0.6444

This suggests that, in this case, changing the dropout rate had minimal impact on model performance.

Observations | Model and Training

- **Batch Size:** 32, 64, or 128

Batch size is a parameter in machine learning and deep learning that determines the number of samples processed before the model's internal parameters are updated. For example, if the batch size is set to 64, the model will process 64 training examples, calculate the loss, and update the model weights once per batch.

- **Smaller batch sizes** (e.g., 16 or 32) can make the model update more frequently and may help it generalize better, but training can be slower.
- **Larger batch sizes** (e.g., 128 or 256) make each update more stable and can speed up training with suitable hardware, but might require more memory and could risk overfitting or poor generalization.

```
# DataLoader
batch_size = 32
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
```

❖ Experiment:

		dropout0.1/0.5		
data	MaxAbs Scaling			
	z_score standardize	y	y	y
classes	12/2 (remap)	2	2	2
Data augmentation	upsampling minority classes			
	Gan1			
	Gan2			
	cGAN			
	dann			
	rnn			
	tGAN			
FCN model	dropout(p)	0.5	0.3	0.3
	early_stopping(patience)			
split?	random split			
	K-Folds			
Batch_size		32	64	128
optimizer(ada)	lr			
	L2()			
loss fct(CrossE)	Compute class weights	y	y	y
Epochs		30	30	30
	Val Acc			
	Test Acc	0.6444	0.6673	0.6304

❖ Result:

There is no significant difference in results when using different batch sizes (32, 64, and 128). The validation and test accuracies for each batch size are as follows:

- Batch size 32: Val_Acc = 0.9551, Test_Acc = 0.6444
- Batch size 64: Val_Acc = 0.9889, Test_Acc = 0.6673
- Batch size 128: Val_Acc = 0.9951, Test_Acc = 0.6304

This suggests that, in this experiment, changing the batch size had only a minor impact on performance.

Observations | Model and Training

- **Split Strategy:** random split, K-Fold

- **Random Split:**

The dataset is randomly divided into training and test (or validation) sets. This is quick and easy, but the results can depend on the random split.

- **K-Fold:**

The dataset is divided into K equal parts (folds). The model is trained K times, each time using a different fold as the validation set and the remaining folds as the training set. The final result is the average of all K runs. This method provides a more reliable estimate of model performance.

❖ Experiment

		split		
data	MaxAbs Scaling			
	z score standardize	y	y	y
classes	12/2 (remap)	2	2	2
Data augmentation	upsampling minority classes			
	Gan1			
	Gan2			
	cGAN			
	dann			
	rnn			
	tGAN			
FCN model	dropout(p)	0.3	0.3	0.3
	early_stopping(patience)			
split?	random split		5(22min)	
	K-Folds			k-Folds
Batch_size		64	64	64
optimizer(adar)	lr			
	L2()			
loss fct(CrossE	Compute class weights	y	y	y
Epochs		30	30	30
	Val_Acc			
	Test_Acc	0.6673	0.6414	0.6344

❖ Result:

The results for the two split strategies (random split and K-Fold) are very similar, and also close to the result without any specific split strategy. The validation and test accuracies for each are as follows:

- Without split: Val_Acc = 0.9889, Test_Acc = 0.6673
- Random split: Val_Acc = 0.9938, Test_Acc = 0.6414
- K-Fold : Val_Acc = 0.9931, Test_Acc = 0.6344

This suggests that, in this case, the choice of split method does not have a significant impact on model performance.

Observations | Model and Training

```
optimizer = optim.Adam(model.parameters(), lr=0.0005, weight_decay=0.01)
```

- **Optimizer:** Adam, learning rate (0.001, 0.0005, 0.005)

• **Learning rate** controls how much the model's weights are updated during each step of the training process. A small learning rate means the model learns slowly and makes small updates, which can help it find a more accurate solution. A large learning rate makes the model learn faster, but if it is too high, the model might not learn properly or could miss the best solution. Choosing an appropriate learning rate is important for effective training

Experiment:

data	MaxAbs Scaling			
	z_score standardize	y	y	y
classes	12/2 (remap)	2	2	2
Data augmentation	upsampling minority classes			
	Gan1			
	Gan2			
	cGAN			
	dann			
	rnn			
	tGAN			
FCN model	dropout(p)	0.3	0.3	0.3
	early stopping(patience)			
split?	random split			
	K-Folds			
Batch_size		64	64	64
optimizer(adar)	lr	0.001	0.0005	0.005
	L2()			
loss fct(CrossE	Compute class weights	y	y	y
Epochs		30	30	30
	Val_Acc			
	Test_Acc	0.6673	0.6484	0.6244

Result:

For the three learning rates tested (0.001, 0.0005, and 0.005), the learning rate of 0.001 gave the best performance, while 0.005 performed the worst. The validation and test accuracies for each learning rate are as follows:

- lr = 0.001: Val_Acc = 0.9889, Test_Acc = 0.6673
- lr = 0.0005: Val_Acc = 0.9963, Test_Acc = 0.6484
- lr = 0.005: Val_Acc = 0.9914, Test_Acc = 0.6244

This indicates that 0.001 is the most suitable learning rate among the three, while both 0.0005 and 0.05 resulted in lower performance.

Observations | Model and Training

`criterion = nn.CrossEntropyLoss(weight=class_weights)`

`criterion = nn.CrossEntropyLoss()`

- **Loss Function:** CrossEntropyLoss , **class weights**

• **Class weights** are used to handle class imbalance in the dataset. If some classes have much fewer samples than others, the model may ignore them and focus on the more frequent classes. By assigning higher weights to the minority classes and lower weights to the majority classes, class weights force the model to pay more attention to underrepresented classes, improving overall performance and fairness in prediction.

❖ Experiment:

data	MaxAbs Scaling		
	z_score standardize	y	y
classes	12/2 (remap)	2	2
Data augmentation	upsampling minority classes		
	Gan1		
	Gan2		
	cGAN		
	dann		
	rnn		
	tGAN		
FCN model	dropout(p)	0.3	0.3
	early_stopping(patience)		
split?	random split		
	K-Folds		
Batch_size		64	64
optimizer{adam}	lr	0.001	0.001
	L2()		
loss fct(CrossE	Compute class weights	y	without
Epochs		30	30
	Val_Acc		
	Test_Acc	0.6673	0.6034

❖ Result:

Without using class weights, the results became worse.
The validation and test accuracies with and without class weights are as follows:

- With class weights:

Val_Acc = 0.9889, Test_Acc = 0.6673

- Without class weights:

Val_Acc = 0.9926, Test_Acc = 0.6034

This shows that using class weights improved the model's performance, especially for imbalanced data.

Observations | Model and Training

- **Epochs:** 20, 30, 40

• **Epochs** refer to the number of times the entire training dataset is passed through the model during training. Each epoch allows the model to learn and adjust its parameters based on all the data. Using more epochs (such as 20, 30, or 40) gives the model more chances to learn patterns in the data. Too few epochs may lead to underfitting, where the model does not learn enough. Too many epochs may cause overfitting, where the model learns the training data too well and performs poorly on new data.

❖ Experiment:

data	MaxAbs Scaling			
	z_score standardize	y	y	y
classes	12/2 (remap)	2	2	2
Data augmentation	upsampling minority classes			
	Gan1			
	Gan2			
	cGAN			
	dann			
	rnn			
	tGAN			
FCN model	dropout(p)	0.3	0.3	0.3
	early_stopping(patience)			
split?	random split			
	K-Folds			
Batch_size		64	64	64
optimizer(adam)	lr	0.001	0.001	0.001
	L2()			
loss fct(CrossE)	Compute class weights	y	y	y
Epochs		30	20	40
	Val_Acc			
	Test_Acc	0.6673	0.6494	0.6104

❖ Result:

Among the epoch settings tested (30, 20, and 40), 30 epochs gave the best results, while 40 epochs performed the worst. The validation and test accuracies for each are as follows:

- 30 epochs: Val_Acc = 0.9889, Test_Acc = 0.6673
- 20 epochs: Val_Acc = 0.9938, Test_Acc = 0.6494
- 40 epochs: Val_Acc = 0.9901, Test_Acc = 0.6104

This suggests that training for 30 epochs achieves the best balance between learning and generalization in this experiment.

Observations | Model and Training

- **Early Stopping:** **early stopping** (patience)

```
def train_model(model, .. early_stop_patience=7):
    if val_accuracy > best_val_accuracy:
        best_val_accuracy = val_accuracy
        best_model_state = model.state_dict()
        patience_counter = 0
    else:
        patience_counter += 1

    if patience_counter > early_stop_patience:
        print(f"Early stopping at epoch {epoch + 1}")
        break
```

• **Early Stopping** is a technique used during model training to prevent overfitting. It stops the training process if the model's performance on the validation set does not improve for a certain number of epochs (called "patience"). This helps the model maintain good generalization to new data.

❖ Experiment:

data	MaxAbs Scaling		
	z_score standardize	y	y
classes	12/2 (remap)	2	2
Data augmentation	upsampling minority classes		
	Gan1		
	Gan2		
	cGAN		
	dann		
	rnn		
	tGAN		
FCN model	dropout(p)	0.3	0.3
	early_stopping(patience)	3min27	1min30s(13)
split?	random split		
	K-Folds		
Batch_size		64	64
optimizer(adar)	lr	0.001	0.001
	L2()		
loss fct(CrossE)	Compute class weights	y	y
Epochs		30	30
	Val Acc		
	Test Acc	0.6673	0.6244

❖ Result:

Using early stopping resulted in a slight drop in accuracy, but significantly reduced the running time—from 3 minutes 27 seconds to 1 minute 30 seconds. Training stopped at epoch 13 when using early stopping. The validation and test accuracies with and without early stopping are as follows:

- Without early stopping:
Val_Acc = 0.9889, Test_Acc = 0.6673
- With early stopping:
Val_Acc = 0.9889, Test_Acc = 0.6244

This shows that early stopping helped save training time, but with a minor decrease in model performance.

Best Model Performances

Best Model Performances

		normalization			dropout0.1/0.5				split															
data	MaxAbs Scaling	Maxabs																						
	z score standardize		z_score	y	y	y	y	y	y	y	y	y	y	y	y	y	y	y	y	y	y	y	y	y
classes	12/2 (remap)	12	12	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
Data augmentation	upsampling minority classes																unsampling							unsampling
	Gan1																gan1(7min)							
	Gan2																	gan2(13min)						
	cGAN																	cgan(17min)						
	dann																		dann(12min)					
	rnn																			rnn(8min)				rnn(8min)
	tGAN																				tgan(13min)			
FCN model	dropout(p)	0.3	0.3	0.3	0.1	0.5	0.3	0.3	0.3	0.3	0.3	0.3	0.3	0.3	0.3	0.3	0.3	0.3	0.3	0.3	0.3	0.3	0.3	0.3
	early stopping(patience)						3min27										1min30s(13)							
split?	random split								5(22min)															
	K-Folds									x-Folds														
Batch_size		32	32	32	32	32	64	128	64	64			64	64	64	64	64	64	64	64	64	64	64	64
optimizer(adar	lr	0.001					0.001						0.001	0.001	0.001	0.001	0.001	0.001	0.001	0.001	0.001	0.001	0.001	0.001
	L2()	0.01										0.0005	0.05											
loss fct(CrossE	Compute class weights	y		y	y	y	y	y	y	y	y	y	y	without	y	y	y	y	y	y	y	y	y	y
Epochs		30		30	30	30	30	30	30	30	30	30	30	30	20	40	30	30	30	30	30	30	30	30
	Val_Acc		0.9408																					
	Test_Acc	0.1049	0.3636	0.6434	0.6454	0.6444	0.6873	0.6304	0.6414	0.6344	0.6484	0.6244	0.6034	0.6494	0.6104	0.6244	0.6314	0.5874	0.5544	0.6404	0.5185	0.6454	0.6234	0.6424

Key Takeaways

Key Takeaways

- 1.Original Data Performs Best:** Training with original data (without augmentation or generative methods) achieved the highest validation and test accuracy, suggesting that the baseline dataset is of high quality for this task.
- 2.Standardization Is Beneficial:** Applying z-score standardization was used consistently and likely contributed to stable training and improved convergence across different experiments.
- 3.Generative Augmentation – Mixed Results:** cGAN and RNN-based augmentation performed almost as well as the original data, showing potential when more data is needed.
- 4.Learning Rate and Optimizer Choices:** The experiments consistently used a learning rate (lr) of 0.001 and Adam optimizer, supporting stable and effective training.
- 5.General Recommendation:** Selecting the right combination of preprocessing (standardization), training settings (epochs, early stopping), and data augmentation methods is crucial for optimal model performance. For your task, simpler approaches (original data, good preprocessing, and careful epoch selection) outperformed more complex augmentation methods.

❖ Future Improvement:

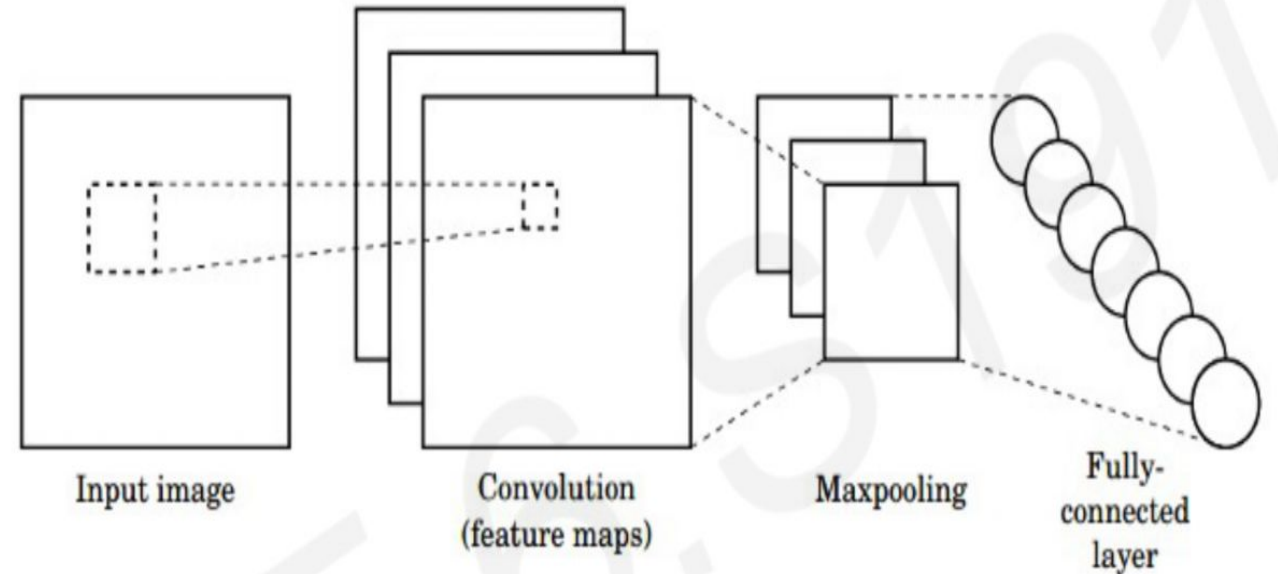
- 1.Information lost** might happen since binary classification was used. Exploring multi-class or multi-label classification, or using more granular labels, could potentially improve model performance and extract richer insights from the data.
- 2.Improving training stability and robustness** should also be focused on, possibly by exploring alternative loss functions, model architectures, or training schedules.
- 3.Other possible improvements** include experimenting with advanced regularization techniques, and additional data augmentation strategies (such as mixup or SMOTE), as well as fine-tuning hyperparameters for further optimization.

2D CNN: Two-Dimensional Convolutional Neural Network

CNN

concept

- Convolutional Neural Networks (CNNs) leverage local connectivity and weight sharing to efficiently learn hierarchical features from grid-structured inputs.
- A 2D CNN applies convolutional kernels over spatial dimensions, making it ideal for extracting patterns from two-dimensional sensor representations.



----- 128 times ----->

2D CNN

System Architecture Overview

- 1.Environment & Seed Setup
- 2.Data Loading & Preprocessing
- 3.Dataset Wrapper
- 4.Model Definition (2D CNN)
- 5.Training Function (train_model)
- 6.Evaluation Function (evaluate_model)
- 7.Visualization Function (plot_training_history)
- 8.Ensemble Methods (Soft Voting / Hard Voting)
- 9.Main Script: 5-fold cross-validation training, model saving, evaluation on validation & test sets, ensemble prediction & analysis
- 10.Metadata Saving: normalization parameters, input shape, etc.

2D CNN

Input Layer :

- 6-axis sensor data (128 timesteps $\times$ 6 channels)
- Converted to 1-channel pseudo-image format

CNN Architecture :

- 3 convolutional blocks with increasing filters (32 $\rightarrow$ 64 $\rightarrow$ 128)
- Batch normalization after each convolution
- ReLU activation for non-linearity
- Max pooling (2 $\times$ 1) for dimensionality reduction
- Dropout (30%) for regularization
- Global average pooling before final classification

Training Components:

Sampling : Weighted random sampling for class imbalance

Optimizer : AdamW with weight decay (L2 regularization)

Loss : Cross entropy with class weighting

Stopping : Early stopping + learning rate reduction

Ensemble Prediction :

- 5-fold cross-validated models
- Soft voting (probability averaging)
- Hard voting (majority decision)

2D CNN

Data Preprocessing

Binary Label Conversion Convert multi-class labels to binary
Normalization

$$\text{normed} = (\text{sensor_array} - \text{means}) / \text{stds}$$

Dataset Splitting

Group-based: Ensure same group (e.g., vehicle ID) not in both sets

Stratified: Maintain class distribution in splits

Dataset Wrapping Convert data to PyTorch tensors
Reshape sensor data

Class Imbalance Handling

Calculate inverse class frequency weights

$$(T, C) \rightarrow (1, T, C)$$

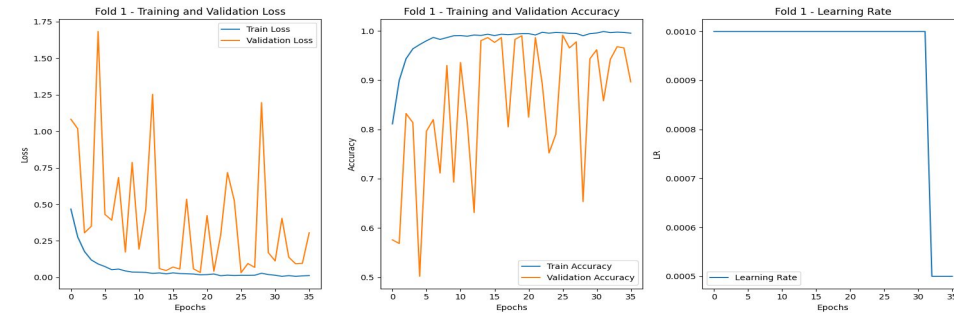
Observations and Comparison

2D CNN-early stopping

Early stopping

10, 20 , no early stopping

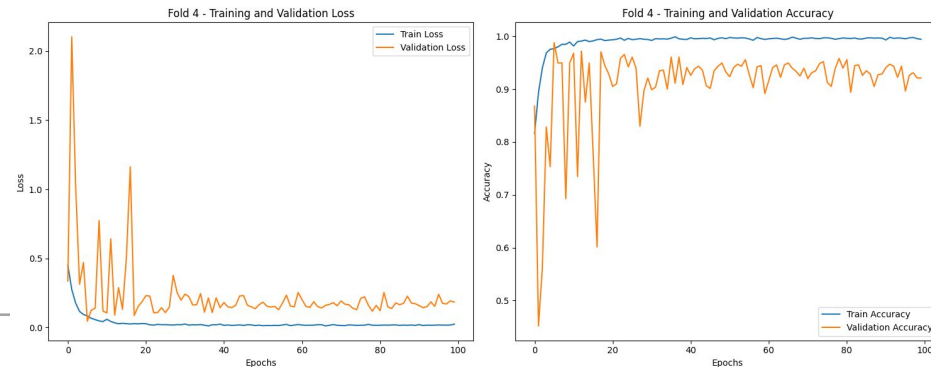
After setting early stopping, the validation curve oscillated, causing the training to stop prematurely even when set to 20 epochs. Therefore, the early stopping mechanism will no longer be used



Patience=10



Patience=20



No early stopping

2D CNN-batchsize

Metric	Batch Size 32	Batch Size 64	Batch Size 128	Batch Size 256	Trend
Validation Loss (↓ better)	0.1024	0.1427	0.2791	0.4680	Continual worsening ↗
Validation Accuracy (↑ better)	96.18%	94.77%	88.85%	80.09%	Continual decrease ↘
Test Set Accuracy	62.14%	60.84%	61.54%	61.74%	Basically stable ↔
Irregular Class F1-score	0.48	0.43	0.40	0.36	Continual decrease ↘
Normal Class Recall	0.75	0.78	0.82	0.86	Continual increase ↗
Irregular Class Recall	0.44	0.36	0.32	0.26	Continual decrease ↘

2D CNN-batchsize

Hidden Problems with Test Performance

Although the test accuracy is stable at ~62%, there's a serious bias:

Abnormal class recall collapses: from 44% → 26% (almost halved)

Normal class recall increases:

from 75% → 86% (model biased towards predicting "Normal")

Large batch size leads the model to ignore learning of minority classes (Abnormal)

F1-score imbalance worsens

Abnormal class F1 drops from 0.48 → 0.36 (↓25%)

Normal class F1 is stable at 0.70-0.73

Proves that the larger the batch size, the worse the model's ability to detect abnormalities.

So we choose batch size=32

2D CNN-LR

Metric	LR=3e-3	LR=1e-2	LR=1e-3	Optimal
Validation Loss (↓)	0.0252	0.0371	0.1024	3e-3
Validation Accuracy (↑)	99.24%	99.21%	96.18%	3e-3
Test Accuracy (↑)	65.03%	61.94%	62.14%	3e-3
Test Macro Avg F1 (↑)	0.64	0.60	0.59	3e-3
Normal Class Recall (↑)	67%	70%	75%	1e-3
Irregular Class Recall (↑)	62%	50%	44%	3e-3
Irregular Class F1 (↑)	0.59	0.52	0.48	3e-3
Recall Gap (Normal-Irregular)	5%	20%	31%	3e-3
Precision-Recall Tradeoff	Balanced	Normal-favored	Normal-favored	3e-3

LR=3e-3 achieves lowest validation loss (0.0252) and highest validation accuracy (99.24%)

All configurations show excellent validation performance (>96%), indicating proper convergence

2D CNN-epoch

Metric	Epoch=50	Epoch=100	Epoch=150	Optimal
Validation Loss (↓)	0.0366	0.0252	0.0248	150
Validation Accuracy (↑)	98.94%	99.24%	99.33%	150
Test Accuracy (↑)	64.94%	65.03%	65.13%	150
Normal Class Precision (↑)	0.70	0.72	0.72	100/150
Normal Class Recall (↑)	0.70	0.67	0.67	50
Irregular Class Precision (↑)	0.57	0.56	0.57	150
Irregular Class Recall (↑)	0.57	0.62	0.62	100/150
Irregular Class F1 (↑)	0.57	0.59	0.59	100/150
Macro F1 (↑)	0.64	0.64	0.64	Equal
Recall Gap (Normal-Irregular)	13%	5%	5%	100/150

2D CNN-epoch

Training Duration Impact

Epoch 50:

Fastest training but lowest irregular class precision (57%)
Most balanced recall but poorest irregular class F1 (0.57)

Epoch 100:

Best irregular class recall (62%)
Optimal precision-recall tradeoff

Epoch 150:

Peak validation performance (99.33% accuracy)
Matches Epoch100 test performance with slight improvements

Epoch150 provides <0.1% test accuracy gain over Epoch100

Requires 50% more training time/resources

2D CNN-weight decay

Metric	WD=1e-5	WD=1e-3	WD=1e-4	Optimal
Validation Loss (↓)	0.0284	0.0277	0.0252	1e-4
Validation Accuracy (↑)	99.19%	99.14%	99.24%	1e-4
Test Accuracy (↑)	64.74%	64.74%	65.03%	1e-4
Normal Class Precision (↑)	0.72	0.72	0.72	Equal
Normal Class Recall (↑)	0.67	0.67	0.67	Equal
Irregular Class Precision (↑)	0.56	0.56	0.56	Equal
Irregular Class Recall (↑)	0.61	0.61	0.62	1e-4
Irregular Class F1 (↑)	0.59	0.59	0.59	1e-4
Macro F1 (↑)	0.64	0.64	0.64	1e-4
Recall Gap (Normal-Irregular)	6%	6%	5%	1e-4

2D CNN-weight decay

Weight Decay	Validation Loss ↓	Validation Accuracy ↑	Test Accuracy ↑	Test F1 (Irregularity) ↑	Test F1 (No Irregularity)	Test Macro F1 ↑	Test Weighted F1 ↑
1e-4	0.0252	0.9924	0.6503	0.59	0.69	0.64	0.65
5e-5	0.0223	0.9933	0.6444	0.58	0.69	0.64	0.65
3e-4	0.0282	0.9921	0.6404	0.57	0.69	0.63	0.64

1e-4 dominates test metrics with best accuracy (65.03%) and irregularity F1 (0.59)

2D CNN-dropout

Dropout Rate	Avg Val Loss	Avg Val Acc	Test Acc	Normal F1	Anomaly F1	Anomaly Recall	Anomaly Precision
0.2	0.0256	99.19%	64.64%	0.70	0.58	60%	56%
0.3	0.0252	99.24%	65.03%	0.69	0.59	62%	56%
0.4	0.0817	97.51%	62.14%	0.70	0.49	45%	54%

Precision-Recall Relationship

Higher dropout improves anomaly precision (54% → 56%) but at severe recall cost (62% → 45%)

Lower dropout maintains recall but reduces precision

Maintain 0.3 dropout as baseline

2D CNN-num_folds

Metric	Fold=8	Fold=5	Fold=3	Observation
Avg Val Loss	0.0243	0.0252	0.0426	Lowest loss with more folds
Avg Val Accuracy	99.48%	99.24%	98.45%	Improves with more folds
Test Accuracy	65.83%	65.03%	65.13%	Best with 8 folds (+0.8% vs fold=5)
Normal F1	0.71	0.69	0.70	Best with 8 folds
Anomaly F1	0.59	0.59	0.57	Consistent across folds
Anomaly Recall	61%	62%	58%	Best with 5 folds
Anomaly Precision	58%	56%	57%	Best with 8 folds

2D CNN-num_folds

Fold Count Impact

Higher folds improve validation metrics:

- Val loss ↓ by 43% (0.0426→0.0243) when increasing from 3→8 folds
- Val accuracy ↑ by 1.03% (98.45% →99.48%)

Test improvements are marginal:

- Only +0.8% test accuracy (65.03% →65.83%) with 3→8 folds
- Anomaly F1 virtually identical (0.57-0.59)

Ensemble Effectiveness

Soft voting ensembles show consistent performance:

- Accuracy range: 65.03%-65.83%
- F1 range: 0.57-0.71

Additional models (8 vs 5) provide <1% accuracy gain

Consider the time, choose num_folds=5

issue

2D CNN-issue-binary label

Model Type	Overall Accuracy	Macro F1	Anomaly/Irr regularity F1	Anomaly Recall	Normal F1	Notes
Multi-Class (12 classes)	49.75%	14.00%	24.75%*	18.25%*	68.00%	Severe class imbalance, 7 classes unused
Multi-Label Binary (4 heads)	76.75%†	56.50%†	27.50%†	45.50%†	84.25%†	RearRight best (F1=0.32, AUC=0.80)
Binary Classification	61.84%	59.00%	48.00%	43.00%	70.00%	Best anomaly detection

Binary labels reduce the task difficulty, leading to a 'false' increase in accuracy.

2D CNN-issue-binary label

Key Observations:

Anomaly Detection Capability:

Binary model leads in irregularity F1 (48% vs multi-label's 27.5%)

Multi-class fails completely on half the classes

Accuracy vs Reality:

Multi-label has highest accuracy (76.75%) but worst anomaly F1 (27.5%)

RearLeft sensor (90% acc) has only 12% irregularity recall

Class Imbalance Impact:

All models show >40% F1 gap between normal and anomaly classes

Multi-class most affected (macro F1=14%)

2D CNN-issue-gan

After loading the generated data, performance remained unchanged, and in some cases (CGAN and DANN), it even declined

Model Type	Avg Val Loss	Avg Val Acc	Test Acc	Normal F1	Anomaly F1	Anomaly Recall
CGAN	0.0351	99.31%	64.84%	0.71	0.56	56%
DANN	0.0256	99.15%	64.64%	0.70	0.58	60%
TGAN	0.0252	99.24%	65.03%	0.69	0.59	62%
No GAN	0.0252	99.24%	65.03%	0.69	0.59	62%

it's also possible that the model is not complex enough or has reached its capacity limit

Best model and future

2D CNN

Metric	Validation (5-Fold Avg)	Test (Soft Voting)	Test (Hard Voting)
Loss	0.0252	-	-
Accuracy	0.9924	0.6503	0.6454
Precision (No Irreg)	-	0.72	0.71
Recall (No Irreg)	-	0.67	0.67
F1 (No Irreg)	-	0.69	0.69
Precision (Irreg)	-	0.56	0.56
Recall (Irreg)	-	0.62	0.61
F1 (Irreg)	-	0.59	0.58
Macro F1	-	0.64	0.64
Weighted F1	-	0.65	0.65

2D CNN future

Next Steps Plan:

- Attempt to use a 1D CNN hybrid architecture.
- Introduce attention mechanism, data augmentation, etc
- Re-attempt using GAN-generated data to address the small data volume issue.
- Re-adjust parameters.

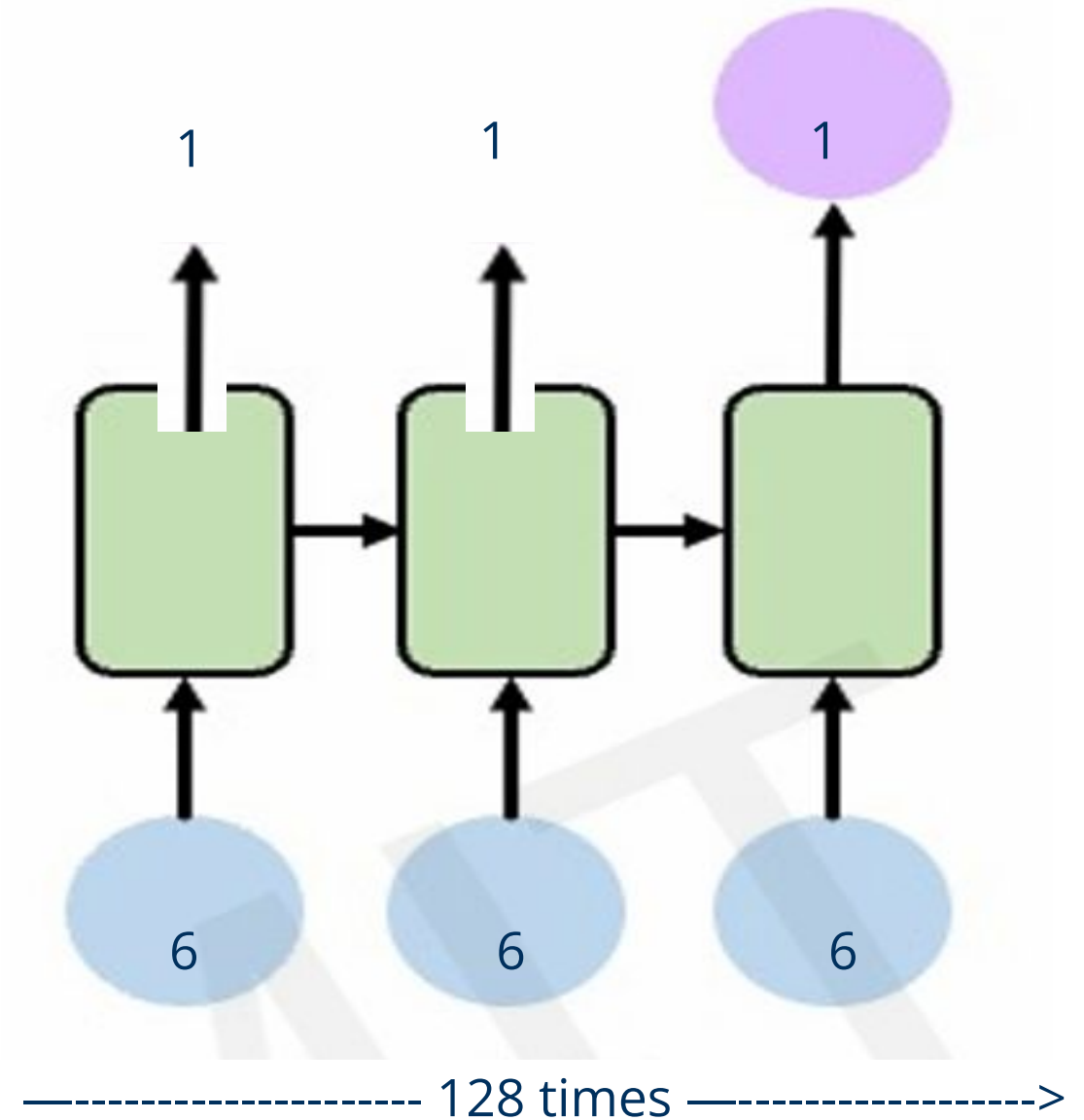
LSTM: Long Short-Term Memory

LSTM

Network Architecture

LSTM as Recurrent Neural Network expects to be trained in multiple iterations.

- 6 Measurement Inputs for 128 iterations
- Logic-Output of dim=1 per iteration
- Translation via sigmoid into probability to be a broken break
- last prediction is taken



LSTM

Data Preprocessing

Using map for labels $x=0,\dots,11$:

$$x \mapsto \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x = 0 \end{cases}$$

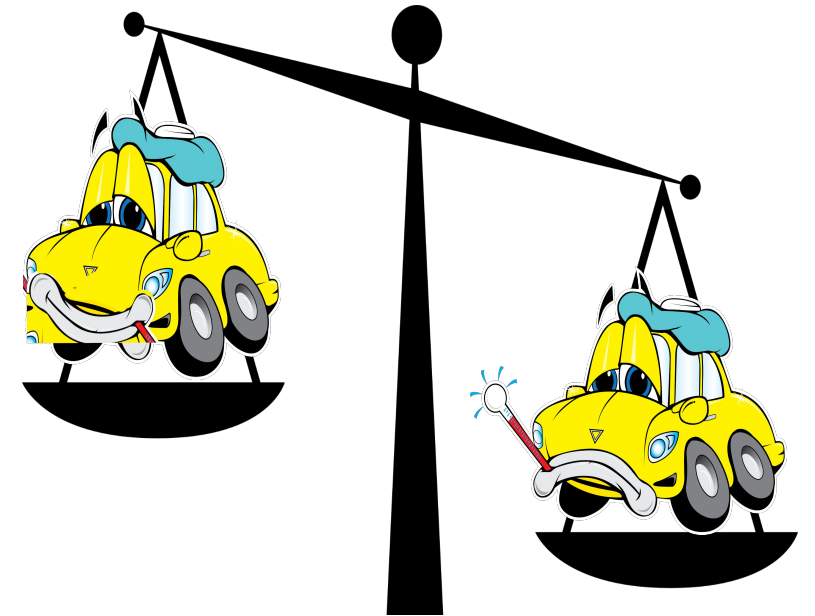
Normalization of measurement Data:

$$AX_{norm} = \left(\frac{ax_1}{ax_{max}}, \dots, \frac{ax_{128}}{ax_{max}} \right), \quad ax_{max} = \max\{|a_1|, \dots, |a_{128}|\}$$

LSTM

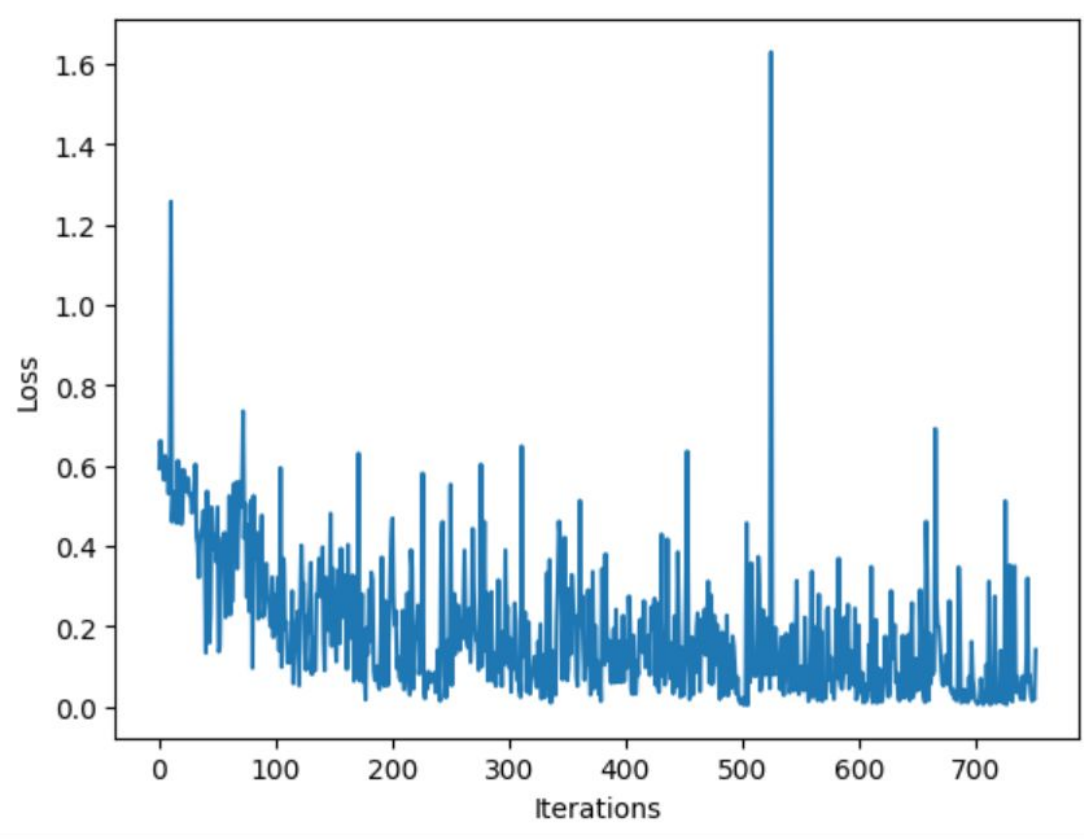
Optimizer, Loss Function and Hyperparameter

- Optimizer: Adam with $lr \in \{10^{-1}, \dots, 10^{-5}\}$ and weight decay
- Loss Function: nn.BCEWithLogitsLoss with/without pos_weight
- Batch Size: $b \in \{8, 16, 32, 64, 128\}$
- Hidden Size: $h \in \{256, 512, 1024, 2048\}$
- Number of Hidden Layer: $n \in \{1, 2\}$ (possibly dropout up to 0.5)



LSTM

Problem of unstable training processes



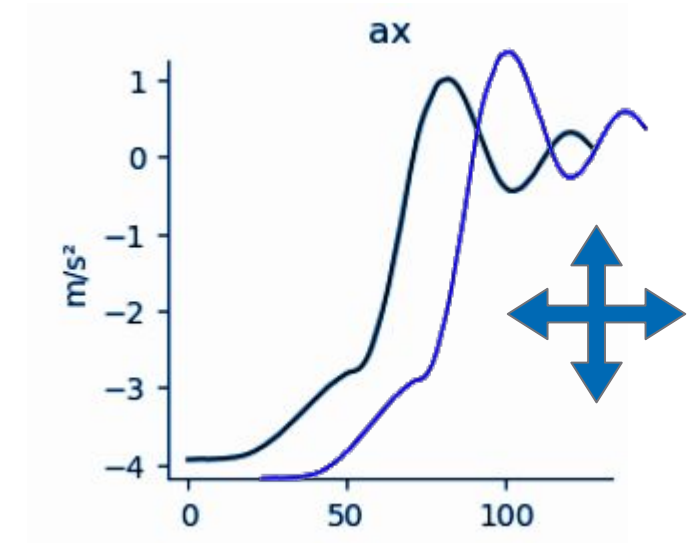
→ bigger batch sizes, e.g. $b = 64$

→ small learning rate, e.g. $lr=10^{-3}$

LSTM

Best Results (fixing batch size, and learning rate)

- Right Label Conversion, no normalization
- nn.BCEWithLogitsLoss with $\text{pos_weight} = \text{wb} / \text{bb} \approx 0.8$
- Single Hidden Layer of size : $h \in 512$
- not using hard coded data augmentation



Setting	Normalization & Label Conversion	No Normalization & No Label Conversion	Weights	Hard Coded Data Augmentation
Accuracy	0.4	0.52	0.56	0.51
Working breaks detected	0	0.49	0.84	0.61
Broken breaks detected	1	0.67	0.36	0.36

LSTM

Best Results (fixing no normalization and use of weights and no data augmentation)

— Trying different learning rates

Setting	$lr=10^{-1}$	$lr=10^{-2}$	$lr=10^{-4}$	$lr=10^{-5}$
Accuracy	0.68	0.68	0.57	0.49
Working breaks detected	0.94	0.93	0.57	0.25
Broken breaks detected	0.3	0.33	0.59	0.84

— Rather big initial steps -> Adam has adaptive step width

LSTM

Best Results - Data Augmentation (fixing batch size, learning rate, no normalization)

Setting	Hard Coded	GAN	RNN	CGAN	TGAN	DANN
Accuracy	0.69	0.7	0.68	0.64	0.69	0.67
Working breaks detected	0.87	0.89	0.83	0.89	0.95	0.87
Broken breaks detected	0.43	0.43	0.47	0.28	0.3	0.38

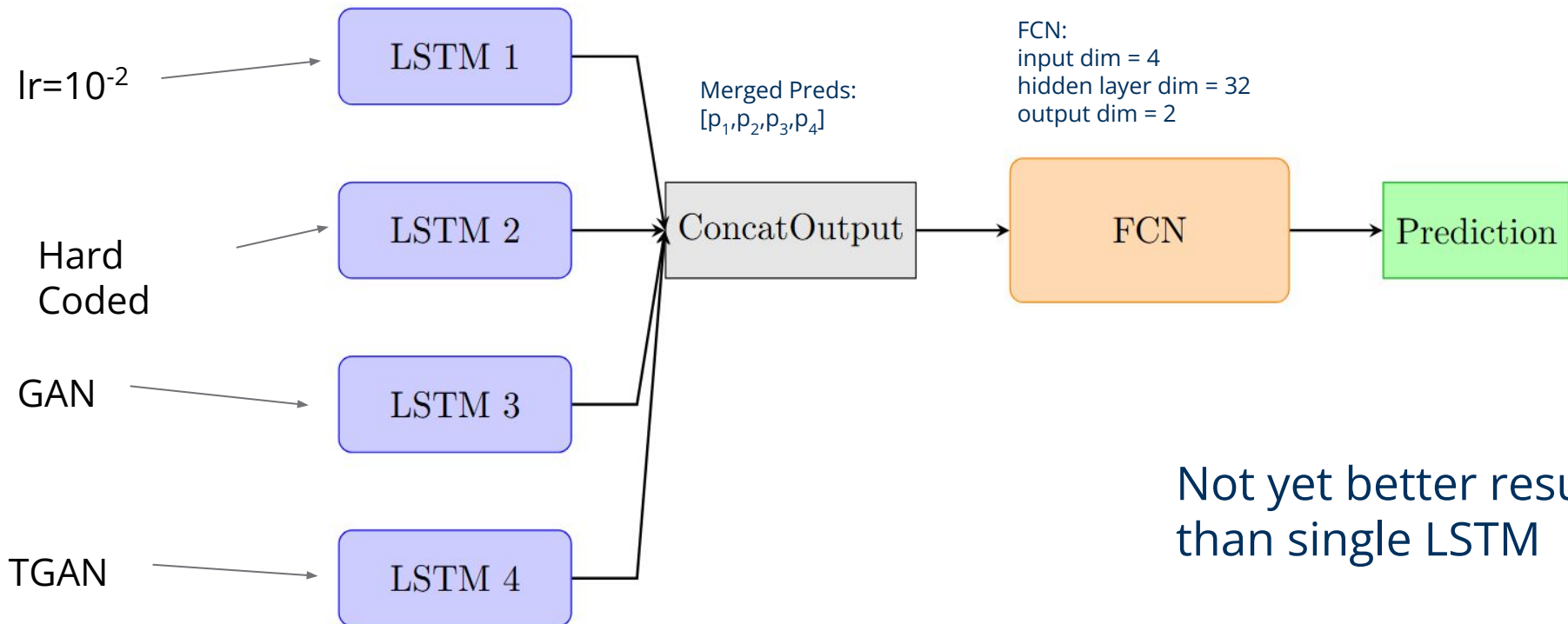
LSTM

Learnings & Outlook

- Bigger networks not necessarily more efficient
- Optimizer and data bias matters!
- Merge all generated data
- Working more on unbiased training data
- Or explain bias to the network (e.g. more information about the car-model)

LSTM

Merging Models



Not yet better result,
than single LSTM :(

Comparison: FCN & 2D CNN & LSTM

Observed Challenges:

❖ FCN:

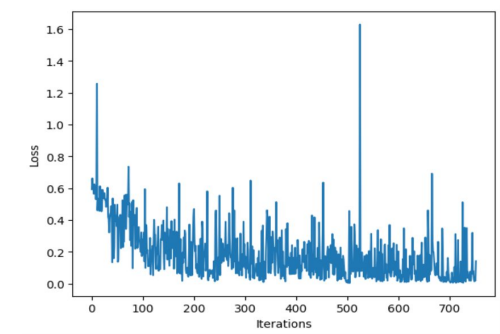
Using binary classification labels led to a loss of information, and the GAN-based data augmentation methods did not provide significant improvements for this architecture.

❖ CNN:

Reduced task difficulty due to binary labels (accuracy decreases when multi-class labels are applied).

Using generated data actually lowers model performance.

❖ LSTM: The main issue was the instability of the training process, making it challenging to achieve consistent results.



Best Model Configuration for Each Architecture:

•FCN (Fully Convolutional Network):

- Used z-score standardization for input features
- Binary classification labels
- Dropout rate: 0.3
- No early stopping (no patience parameter)
- No random split for data partitioning
- Batch size: 64
- Optimizer: Adam with learning rate 0.001
- L2 regularization (weight decay): 0.01
- Loss function: CrossEntropyLoss with class weights
- Number of epochs: 30

• 2DCNN (2D Convolutional Neural Network):

- Set random seeds and convert original multi-class labels to binary labels. .
- Global Normalization
- K-Fold Cross-Validation
- Ensemble Learning: Soft voting and hard voting
- Loss Function: CrossEntropyLoss
- Activation Function: Rectified Linear Unit
- Batch size: 32
- Learning rate: 3e-3
- epochs: 100
- No early stopping
- weight decay :1e-4

•LSTM (Long Short-Term Memory Network):

- Applied right label conversion (no normalization of inputs)
- Loss function: nn.BCEWithLogitsLoss with pos_weight ≈ 0.8 (calculated as w_b / b_b)
- Single hidden layer with 512 units
- Used hard-coded data augmentation or GAN-based augmentation
- Learning rate: 0.1 (10^{-1})

Accuracy of Best Model:

0.67

0.65

0.70

Task 2: Generative AI

GAN, RNN, Transformer-GAN, DANN

Problems We Want to Tackle

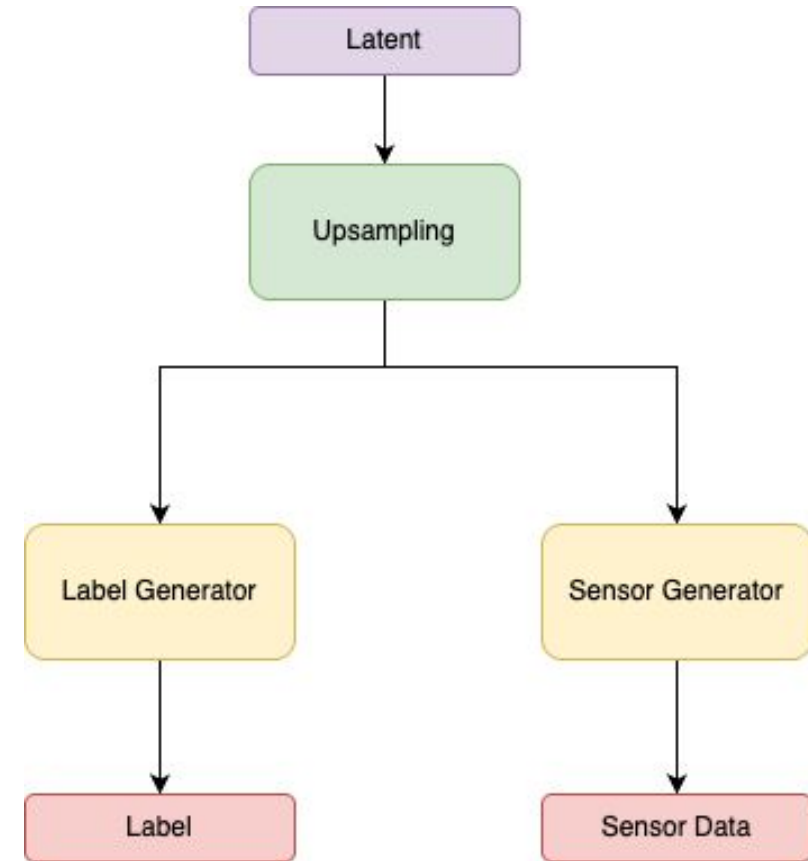
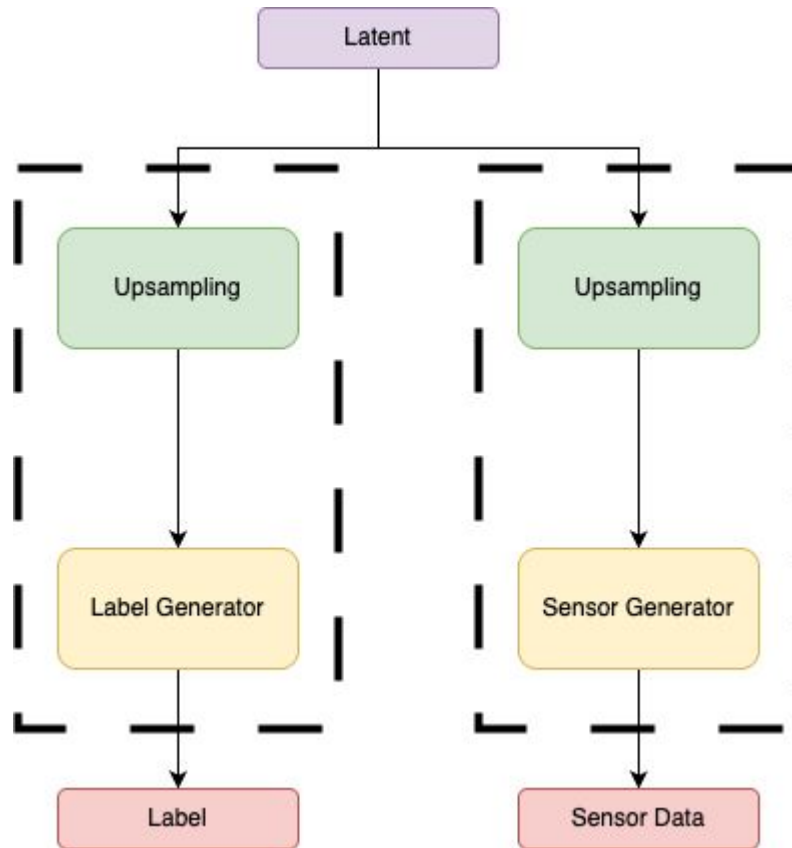
Implemented basic data generation using GAN and RNN models

Developed cGAN models to address training set label imbalance

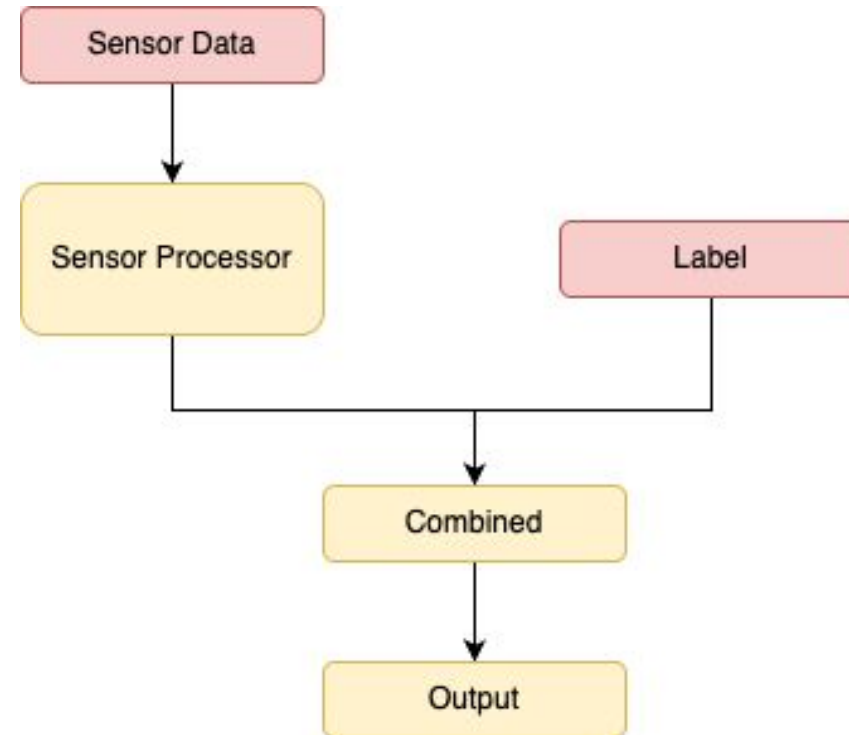
Developed DANN models to handle distribution mismatches between training and test sets

Implemented transformer-based GANs to capture more complex temporal patterns

GAN-Generator

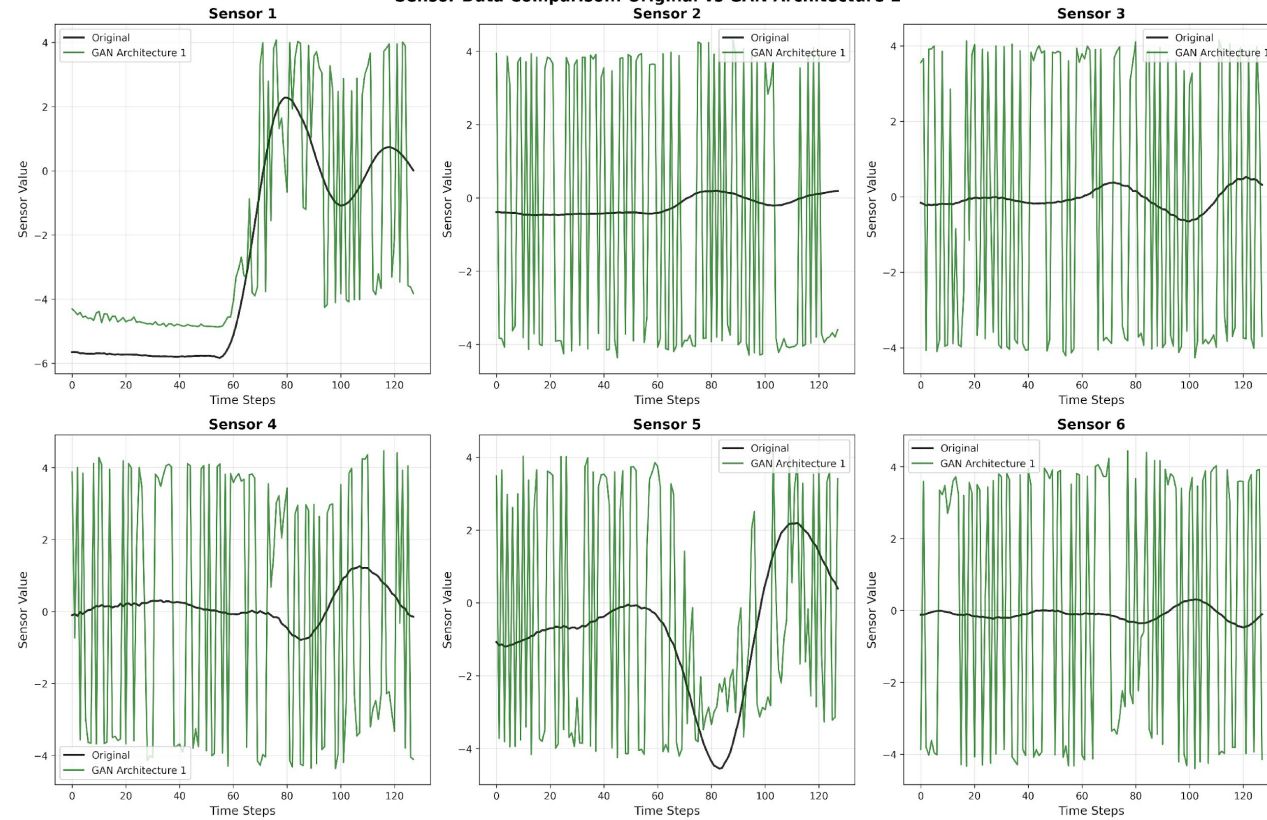


GAN-Discriminator

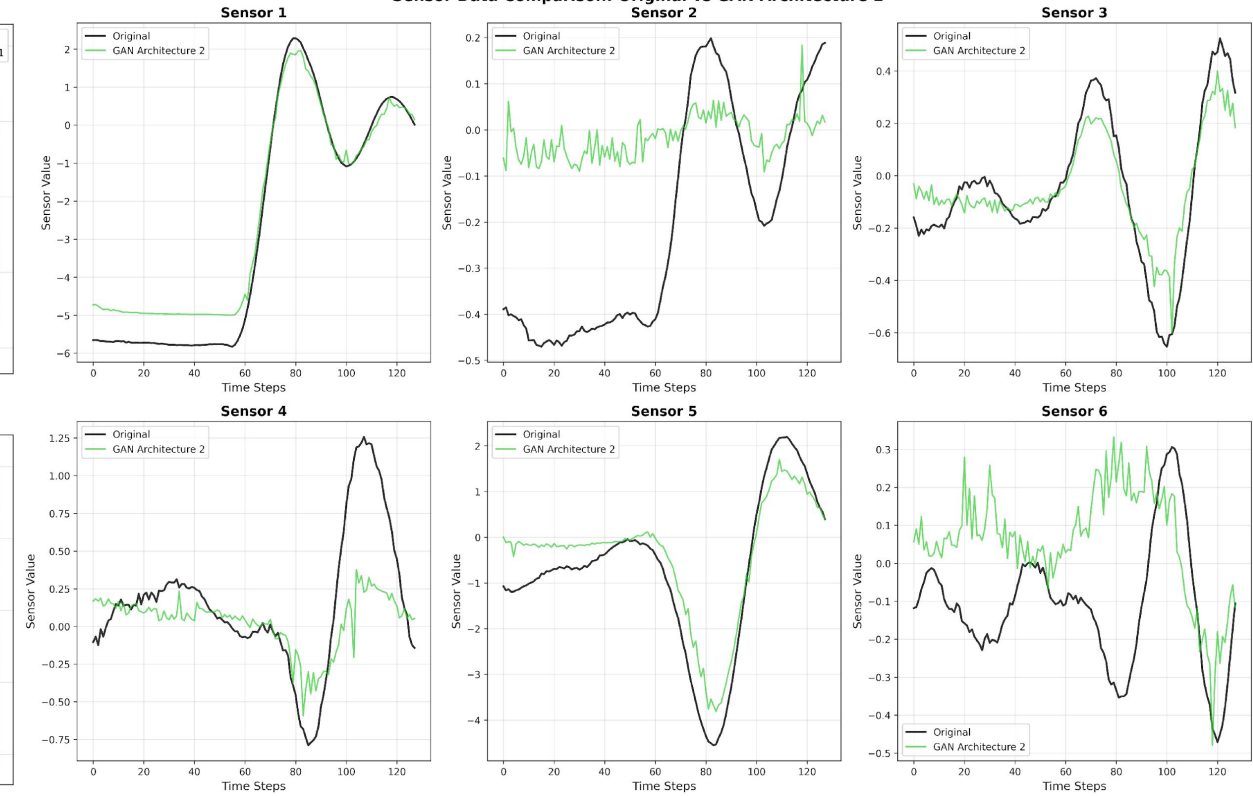


GAN

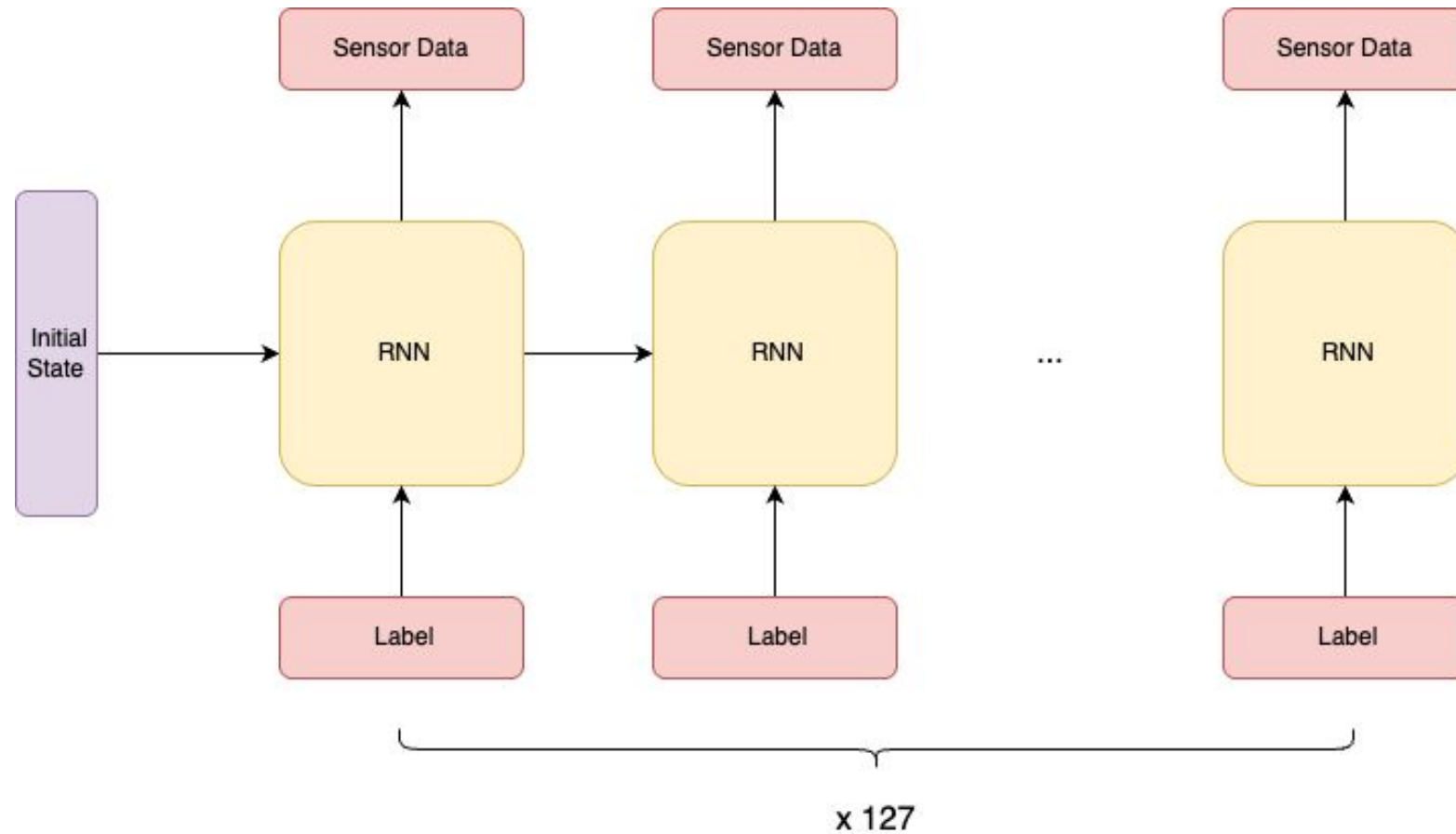
Sensor Data Comparison: Original vs GAN Architecture 1



Sensor Data Comparison: Original vs GAN Architecture 2



RNN



RNN

Sensor Data Comparison: Original vs RNN Generated

